

Programozás Alapjai

Dr. Gergely Tamás
Dr. Kicsi András

Szegedi Tudományegyetem
Informatikai Intézet
Szoftverfejlesztés Tanszék

2025

(v1207)



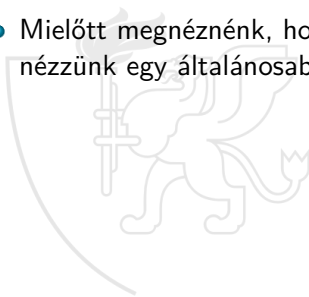
Tartalomjegyzék

- 
- 1 Bemutatók**
 - Kurzus információk
 - Infrastruktúra szabályzatok
 - Mentorálás
 - 2 Python alapok**
 - Bevezető
 - Pénzváltás (1. verzió)
 - Pénzváltás (2. verzió)
 - 3 Alapfogalmak**
 - Alapfogalmak
 - Alapvető programelemek fogalmak
 - 4 Python feltételes ciklusok**
 - Röppálya számítás
 - Röppálya szimuláció
 - 5 Szoftverfejlesztési életciklus modellek**
 - A programozás fázisai
 - 6 Python sztringek**
 - Python sztringek
 - Basename meghatározása
 - 7 Vezérlési szerkezetek**
 - Bevezetés
 - Szekvenciális vezérlés
 - Függvények
 - 8 Python listák**
 - Python list és tuple adattípusok
 - Python listáshoz adattípusok műveletei
 - Basename és eltelt idő
 - 9 Vezérlési szerkezetek**
 - Szelekciós vezérlések
 - 10 Python iterációk**
 - Csúszólagos számítás
 - 11 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 12 Python halmazok, comprehension**
 - Python set és frozenset adattípusok
 - Tér legtávolabbi pontjai
 - 13 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 14 Python dictionary, függvény mint típus**
 - Python dict adattípus
 - Python comprehension
 - Python function adattípus
 - 15 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 16 Python fájlkezelés**
 - Fájlkezelés
 - 17 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 18 Python formátált IO**
 - Python formátált IO
 - 19 Adatkezelés, adattárolás**
 - Az adatkezelés szintjei
 - Az adattárolás jellemzői
 - 20 Python típusok**
 - Python numerikus típusok
 - 21 Absztrakt adattípusok**
 - Egyszerű adattípusok
 - Összetett adattípusok
 - 22 Python típusok**
 - További Python típusok
 - Python function adattípus
 - 23 Absztrakt adattípusok**
 - Összetett adattípusok
 - 24 Tesztelés**
 - Tesztelés
 - Tesztelés Python-ban
 - A pytest
 - 25 Vizualizáció és egyéb modulok**
 - Fájlkezelő modulok
 - Vizualizáció
 - 26 More Python**
 - A Python program végrehajtásának folyamata
 - Függvényparaméterek és argumentumok
 - Python unpacking
 - Kivételkezelés

Tartalomjegyzék

- 
- 1 Bemutatók**
 - Kurzus információk
 - Infrastruktúra szabályzatok
 - Mentorálás
 - 2 Python alapok**
 - Bevezető
 - Pénzváltás (1. verzió)
 - Pénzváltás (2. verzió)
 - 3 Alapfogalmak**
 - Alapfogalmak
 - Alapvető programelemek fogalmak
 - 4 Python feltételes ciklusok**
 - Röppálya számítás
 - Röppálya szimuláció
 - 5 Szoftverfejlesztési életciklus modellek**
 - A programozás fázisai
 - 6 Python sztringek**
 - Python sztringek
 - Basename meghatározása
 - 7 Vezérlési szerkezetek**
 - Bevezetés
 - Szekvenciális vezérlés
 - Függvények
 - 8 Python listák**
 - Python list és tuple adattípusok
 - Python listáshoz adaptívusok műveletei
 - Basename és eltelt idő
 - 9 Vezérlési szerkezetek**
 - Szelekciós vezérlések
 - 10 Python iterációk**
 - Csúszóátlag számítása
 - 11 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 12 Python halmazok, comprehension**
 - Python set és frozenset adattípusok
 - Tér legtovábbi pontjai
 - 13 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 14 Python dictionary, függvény mint típus**
 - Python dict adattípus
 - Python comprehension
 - Python function adattípus
 - 15 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 16 Python fájlkezelés**
 - Fájlkezelés
 - 17 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 18 Python formátált IO**
 - Python formátált IO
 - 19 Adatkezelés, adattárolás**
 - Az adatkezelés szintjei
 - Az adattárolás jellemzői
 - 20 Python típusok**
 - Python numerikus típusok
 - 21 Absztrakt adattípusok**
 - Egyszerű adattípusok
 - Összetett adattípusok
 - 22 Python típusok**
 -
 - 23 Absztrakt adattípusok**
 - További Python típusok
 - Python function adattípus
 - Összetett adattípusok
 - 24 Tesztelés**
 - Tesztelés
 - Tesztelés Python-ban
 - A pytest
 - 25 Vizualizáció és egyéb modulok**
 - Fájlkezelő modulok
 - Vizualizáció
 - 26 More Python**
 - A Python program végrehajtásának folyamata
 - Függvényparaméterek és argumentumok
 - Python unpacking
 - Kivételkezelés

- Eljárásvezérlésről akkor beszélünk, amikor egy műveletet adott argumentumokra alkalmazunk, aminek hatására az argumentumok értékei pontosan meghatározott módon változnak meg.
- Az eljárásvezérlés fajtái:
 - Eljárásművelet
 - Függvényművelet
- Mielőtt megnéznénk, hogy Python nyelven hogyan valósulnak meg ezek a műveletek, nézzünk egy általánosabb definíciót.



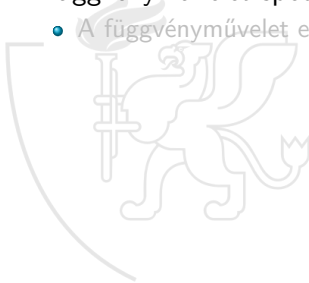
- Eljárasműveleten olyan tevékenységet értünk, amelynek alkalmazása adott argumentumokra az argumentumok értékének pontosan meghatározott megváltozását eredményezi.
- Minden eljárasműveletnek rögzített számú paramétere van, és minden paraméter rögzített adattípusú.
- Minden paraméter rögzített kezelési módú, ami lehet:
 - Bemenő mód
 - Ha a művelet végrehajtása nem változtathatja meg az adott argumentum értékét.
 - Kimenő mód
 - Ha a művelet eredménye nem függ az adott argumentum végrehajtás előtti értékétől, de az adott argumentum értéke a művelet hatására megváltozhat.
 - Be- és kimenő (vegyes) mód
 - Ha a művelet felhasználhatja az adott argumentum végrehajtás előtti értékét és az argumentum értéke a művelet hatására meg is változhat.

- A matematikai függvény fogalmának általánosítása.
- Ha egy részprobléma célja egy érték kiszámítása adott értékek függvényében, akkor a megoldást megfogalmazhatjuk függvényművelettel.
- A függvényművelet argumentumai ugyanúgy lehetnek kimenő és be- és kimenő módúak is, mint az eljárasműveletek esetén, tehát a függvényművelet végrehajtása az aktuális argumentumok megváltozását is eredményezheti.



Eljárás és függvény specifikációja

- Az eljárasművelet specifikációja tartalmazza:
 - A művelet elnevezését
 - A paraméterek felsorolását
 - Mindegyik paraméter adattípusát
 - Mindegyik paraméter kezelési módját
 - A művelet hatásának leírását
- A függvényművelet specifikációja a fentiekén túl tartalmazza még:
 - A függvényművelet eredménytípusát



Eljárásművelet általános jelölése

- Eljárásműveletet $P(m_1 X_1 : T_1; \dots; m_n X_n : T_n)$ formában jelölhetünk, ahol
 - P az eljárásművelet neve
 - m_i az i . paraméter kezelési módja
 - X_i az i . paraméter azonosítója
 - T_i az i . paraméter adattípusa
- Az eljárás algoritmusát egy olyan struktogrammal adható meg, melynek a feje így néz ki:



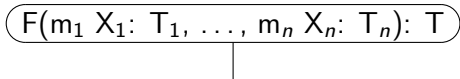
$P(m_1 X_1: T_1, \dots, m_n X_n: T_n)$

- A fent jelölt eljárasművelet adott $A_1, \dots, A_n$ argumentumokra történő végrehajtását eljárashívásnak nevezzük és a $P(A_1, \dots, A_n)$ jelölést használjuk.
- Ha az i . paraméter kezelési módja kimenő vagy be- és kimenő, akkor az A_i aktuális argumentum csak változó lehet.
- Az eljárashívás utasítás.



Függvényművelet általános jelölése

- Függvényművelet $F(m_1 X_1 : T_1; \dots; m_n X_n : T_n) : T$ formában jelölhetünk, ahol
 - F a függvényművelet neve
 - m_i az i . paraméter kezelési módja
 - X_i az i . paraméter azonosítója
 - T_i az i . paraméter adattípusa
 - T a függvényművelet eredménytípusa
- A függvény algoritmusát egy olyan struktogrammal adható meg, melynek a feje így néz ki:


$$F(m_1 X_1: T_1, \dots, m_n X_n: T_n): T$$

- Továbbá a struktogramon lennie kell (legalább) egy olyan utasításnak, amely a függvény aktuális argumentumokra számított értékével visszatér a függvényből.

- A fent jelölt függvényműveletnek adott $A_1, \dots, A_n$ argumentumokra történő végrehajtását függvényhívásnak nevezzük és az $F(A_1, \dots, A_n)$ jelölést használjuk.
- Ha az i . paraméter kezelési módja kimenő vagy be- és kimenő, akkor az A_i aktuális argumentum csak változó lehet.
- A függvényhívás kifejezés.



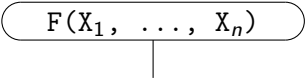
Eljárásvezérlés megvalósítása

Megvalósítás Python nyelven

- Vannak olyan programozási nyelvek, ahol a függvény és eljárasműveletek meg vannak különböztetve, a típusokat fordítási időben meg kell adni, valamint a paraméterek módjaira sincs megkötés.
- Mivel azonban e kurzus keretében csak a Python nyelvről lesz szó, a struktogramon a továbbiakban (legalábbis a legtöbb esetben) igazodni fogunk a Python nyelvhez.
 - A Python nyelvben lényegében csak függvényművelet van.
 - A Python nyelvben nem a programelemeknek, hanem az értékeknek van típusuk.
 - Python nyelvben a függvényművelet argumentumai bemenő módon referenciaként adódnak át, ami miatt a függvényművelet végrehajtása az aktuális argumentumok értékének megváltozását eredményezheti (ha a referált objektum változtatható (*mutable*)).

Függvényművelet

- A függvényművelet jelölésére a $F(X_1, \dots, X_n)$ formát használjuk, ahol
 - F a függvényművelet neve
 - X_i az i . paraméter azonosítója
- A zárójeleket üres paraméterlista esetén is ki kell tenni.
- A Python jelölésmódhoz igazodva, a függvény algoritmusát egy olyan struktogrammal adható meg, melynek a feje így néz ki:



$F(X_1, \dots, X_n)$

- Továbbá a struktogramon lennie kell (legalább) egy olyan `return` utasításnak, amely visszaadja a függvény által kiszámított értéket.

Függvényművelet

- A fent jelölt függvényműveletnek adott $A_1, \dots, A_n$ aktuális argumentumokra történő végrehajtását függvényhívásnak nevezzük és az $F(A_1, \dots, A_n)$ jelölést használjuk.
- A függvényhívás kifejezés.
- A zárójeleket paraméter nélküli függvény hívása esetén is ki kell tenni.



A return utasítás

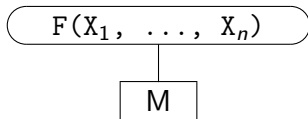
- Minden függvényben lehet egy vagy több return utasítás.
- Ha a függvényben egy ilyen utasítást hajtunk végre, akkor a függvény értékének kiszámítása befejeződik. A hívás helyén a függvényhívás kifejezés a return által kiszámított értéket veszi fel.
- Ha egy függvényben nincs return utasítás, vagy a függvény nem ennek végrehajtásával ér véget, az olyan, mintha egy return None-t hajtottunk volna végre.



Függvényművelet

Megvalósítás Python nyelven

- Struktogramja:



- Megvalósítása:

```
def F(X1, ..., Xn):  
    M
```



- Ha eljárást szeretnénk készíteni Python nyelven, akkor egy olyan függvényt kell deklarálni, mely nem ad vissza értéket.
- Technikailag ez is egy függvény, ami a `NoneType` típusú `None` értéket adja vissza.



- A Python nyelvben egy változónak háromféle láthatósága (scope-ja) lehet:
 - `local` Függvényeken belül ez a default: a függvényen belül értéket kapó azonosító csak a függvényen belül használható.
 - `global` Függvényeken kívül lehet: a függvényen kívül értéket kapó azonosító – azt esetleges törléséig – a programban bárhol elérhető lehet (ha egy lokális változó egy függvényen belül *el nem fed*i).
 - `nonlocal` Függvényen belül deklarált függvényben a „külső” függvény azonosítói ilyen módon érhetők el a „belső” függvényből.
- A Python nyelvben függványszintű scope-ok vannak: egy változó egy adott függvényen belül (nem feltétlenül számítva az azon belül deklarált „belső” függvényeket) „mindenhol” vagy használható, vagy nem használható.

- Sorrendiségi szabály
 - Egy változónak az adott scope-ban mindenképpen értéket kell adnunk, mielőtt először használnánk. A változó ugyanis az első (végrehajtott) értékadással jön létre.
- Egyediségi szabály
 - Egy adott scope-ban egy azonosító nem jelölhet több különböző programelemet.
- Láthatósági szabály
 - A lokális azonosító a saját scope-ján (és az alárendelt, beágyazott scope-okon) kívül nem létezik.
 - A globális és nem-lokális azonosítók belsőbb scope-okból kétféleképpen hivatkozhatók:
 - Read-only módon, azaz nem kaphatnak értéket. (Ha egy scope-ban egy változó bárhol értéket kap, akkor alaphelyzetben új lokális változónak minősül, és érvénybe lép a sorrendiségi szabály.)
 - A használat előtt a `global` illetve `nonlocal` kulcsszavakkal deklarálni kell őket, ekkor lehetőségünk van új értéket is adni számukra.

Változók láthatósága a Python nyelvben

Sorrendiség

```
>>> print(f'{{gv}}')
```

Traceback (most recent call last):
 File "<stdin>", line 1, in <module>
NameError: name 'gv' is not defined

```
>>> gv = "Globális"  
>>> print(f'{{gv}}')
```

'Globális'



Változók láthatósága a Python nyelvben

Lokális változó

```
>>> def f():  
...     lv = "Lokális"  
...     print(f"f(): {lv}")  
...  
>>> f()  
f(): 'Lokális'  
>>> print(f" '{lv}' ")  
Traceback (most recent call last):  
  File "<stdin>", line 1, in <module>  
NameError: name 'lv' is not defined
```

Változók láthatósága a Python nyelvben

Globális változóra való read-only hivatkozás

```
>>> var = "Globális"
>>> def f():
...     print(f"f():_{var}")
...
>>> f()
f(): 'Globális'
>>> print(f"_{var}")
'Globális'
```

Változók láthatósága a Python nyelvben

Lokális változó „elfedi” a globális változót

```
>>> var = "Globális"
>>> def f():
...     var = "Lokális"
...     print(f"f(): {var}")
...
>>> f()
f(): 'Lokális'
>>> print(f" '{var}' ")
'Globális'
```

Változók láthatósága a Python nyelvben

Lokális változó okozta sorrendiségi hiba

```
>>> var = "Globális"
>>> def f():
...     print(f"f():_ '{var}'")
...     var = "Lokális"
...
>>> f()
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "<stdin>", line 2, in f
UnboundLocalError: local variable 'var' referenced before assignment
>>> print(f" '{var}'")
'Globális'
```

Változók láthatósága a Python nyelvben

Globális változó teljes elérése

```
>>> var = "Globális"
>>> def f():
...     global var
...     print(f"f(before):_{var}")
...     var = "Továbbra is az"
...     print(f"f(after):_{var}")
...
>>> f()
f(before): 'Globális'
f(after): 'Továbbra is az'
>>> print(f"_{var}")
'Továbbra is az'
```

Változók láthatósága a Python nyelvben

Lokális, Nem lokális, Globális

```
>>> var = "Globális_érték"
>>> def f():
...     var = "Külső_érték"
...     def g():
...         var = "Belső_érték"
...         g()
...         print(f"f(): {var}")
...
>>> f()
f(): 'Külső érték'
>>> print(f" '{var}' ")
'Globális érték'
```

Változók láthatósága a Python nyelvben

Lokális, Nem lokális, Globális

```
>>> var = "Globális"
>>> def f():
...     var = "Külső_érték"
...     def g():
...         nonlocal var
...         var = "Belső_érték"
...     g()
...     print(f"f():_{var}")
...
>>> f()
f(): 'Belső érték'
>>> print(f"_{var}")
'Globális érték'
```

Változók láthatósága a Python nyelvben

Lokális, Nem lokális, **Globális**

```
>>> var = "Globális"
>>> def f():
...     var = "Külső_érték"
...     def g():
...         global var
...         var = "Belső_érték"
...     g()
...     print(f"f():_{var}")
...
>>> f()
f(): 'Külső érték'
>>> print(f"_{var}")
'Belső érték'
```

Argumentumkezelés Python nyelven

- Egy Python függvény paraméterei a függvényen belül lokális változóként működnek. (Olyan lokális változóként, amely a függvény meghívásakor kap kezdőértéket.)
- A Python-ban referencia szerinti argumentumátadás van, így a *formális paraméter értékén* végzett műveletek kihatnak az aktuális argumentum értékére.
- A Python-ban a referencia átadása bemenő módon történik, így a formális paraméternek történő értékadásnak nincs hatása az aktuális argumentumra.

```
>>> def f(x):  
...     x[0] = 'Hello!'  
...     del x[-1]  
...  
>>> a = [0,1]  
>>> f(a)  
>>> a  
['Hello!']
```

```
>>> def f(x):  
...     x = x[::-1]  
...     print(x)  
...  
>>> a = "Hello!"  
>>> f(a)  
!olleH  
>>> a  
'Hello!'
```